



# EMERGENCY SUB PLANS PACK

for DIGITAL TECHNOLOGIES

★ Keep learning on track—even when plans change. ★



**NO-PREP  
ACTIVITIES**



**READY-TO-USE  
RESOURCES**



**PERFECT FOR  
EMERGENCY  
LESSONS**



**GREAT FOR  
SUBSTITUTE  
TEACHERS**



**ENGAGING  
DIGITAL TECH  
TASKS**



**PRINT OR  
DIGITAL USE**



YOUR **LIFESAVER** IN THE CLASSROOM



SAVE TIME



STAY PREPARED



KEEP STUDENTS  
ENGAGED



MADE BY TEACHERS  
FOR TEACHERS

# Set this up while you are well

15 MINUTES

You are reading this because you are **well**. Spend fifteen minutes now and this pack will run itself on a day when you are not.

## The one job you have today

Print pages 3–6 of this pack and leave them in a clearly labelled folder where your substitute teacher will find them – top drawer, on the desk, or with the front office. Everything a substitute needs is in those four pages.

You do not need to print the lessons in advance. The substitute teacher chooses one on the day and photocopies it.

## SETTING IT UP • FIFTEEN MINUTES, ONCE

- 1 **Print the whole pack once.** Put it in a display folder or bind it. Write your name and room number on the cover.
- 2 **Print pages 3–6 a second time** and clip them to the front. These are the pages a substitute teacher reads first.
- 3 **Tell your faculty head where it lives.** On a genuine emergency day you will not be answering your phone, so somebody else needs to know.
- 4 **Cross off the lessons you have already used.** There is a tracker on page 6. Two years of substitute days will not repeat if you keep it current.
- 5 **Optional: leave a class list** and a note about any students who need specific support, seating or a reader.

### Why these lessons are unplugged

On a substitute day the laptop trolley is often locked, the login server is down, or nobody knows the room password. Every lesson here runs on paper.

Each lesson also has an **If the computers are working** note, so the same sheet still works when technology cooperates.

### Why they are not busy work

Every lesson targets a genuine Digital Technologies concept and produces marked evidence you can put in a folder.

If you are away for a week, running four of these in sequence is a defensible unit of work, not a holding pattern.

### A word about the answer keys

Every answer key sits on the **last page of its lesson**. If you hand a lesson to a substitute teacher, hand them the answer key too – it is the single thing that lets a non-specialist mark with confidence and answer a student who says “*but why?*”

# You do not need to know this subject

You have walked into a Digital Technologies class. You may have never taught the subject. That is completely fine — this pack was written for you.

## Read this and you are ready

**You do not need to know how to code.** Every lesson is a paper worksheet with a full answer key. Your job is to hand it out, keep the room settled, and check work as students finish. You are not expected to teach the content.

## DO THESE FOUR THINGS

- 1 **Pick one lesson** from the table on the next page. Any of them works for any class from Year 7 to Year 10. If you cannot decide, use Lesson 05.
- 2 **Photocopy the student pages only.** Each lesson tells you exactly which page numbers to copy. One copy per student.
- 3 **Keep the teacher page and the answer key.** Do not photocopy those.
- 4 **Read the teacher page once** before the bell. It takes about two minutes and includes the exact words you can say to start the lesson.

### If a student says “we have done this”

Ask them to prove it by completing the extension section at the end. If they genuinely have, switch them to a different lesson from the table — the pack has ten, and they will not have done them all.

### If you finish early

Every lesson has an extension task on its final student page. Point students there. There is also a discussion prompt on the teacher page you can run as a whole class for the last five minutes.

## What to leave for the regular teacher

Collect the completed sheets. Fill in the handover note on the last page of this pack — it takes ninety seconds and tells the regular teacher which lesson you used, how far the class got, and anything that happened.

Leave both on the desk.

# Choose your lesson

10 OPTIONS

Every lesson is standalone. There is no order and no prerequisite — pick whichever suits the class in front of you.

| #  | LESSON                             | WHAT IT COVERS                   | DIFFICULTY | MINS | STUDENT PAGES |
|----|------------------------------------|----------------------------------|------------|------|---------------|
| 01 | Follow My Instructions Exactly     | Algorithms & precision           | ★          | 60   | 7–9           |
| 02 | Counting Like a Computer           | Binary & data representation     | ★★         | 60   | 12–14         |
| 03 | Trace the Code                     | Reading programs by hand         | ★★         | 55   | 17–19         |
| 04 | Flowchart Detective                | Logic & decisions                | ★          | 55   | 22–24         |
| 05 | How Attacks Actually Work          | Cyber security                   | ★          | 60   | 27–29         |
| 06 | Sending a Message Across the World | Networks & the internet          | ★★         | 60   | 32–34         |
| 07 | Sorting and Searching by Hand      | Algorithm efficiency             | ★★★        | 60   | 37–39         |
| 08 | Bug Hunt                           | Debugging & error types          | ★★★★       | 55   | 42–44         |
| 09 | What AI Can and Cannot Do          | Artificial intelligence & ethics | ★          | 60   | 47–49         |
| 10 | Your Digital Footprint             | Privacy & digital citizenship    | ★          | 55   | 52–54         |

## Easiest to supervise

**05, 09 and 10.** Scenario and discussion based. Students have opinions, so the room stays busy and nobody gets stuck.

## Best for a quiet, working class

**02, 03 and 07.** Puzzle-like with definite answers. Strong students will race; that is what the extensions are for.

## Best for a restless class

**01 and 04.** Partner activities with movement and a bit of humour built in. Noisier, but productively so.

## Used-lesson tracker • for the regular teacher

| LESSON                 | DATE USED | CLASS | NOTES |
|------------------------|-----------|-------|-------|
| 01 – Instructions      |           |       |       |
| 02 – Binary            |           |       |       |
| 03 – Trace the Code    |           |       |       |
| 04 – Flowcharts        |           |       |       |
| 05 – Cyber security    |           |       |       |
| 06 – Networks          |           |       |       |
| 07 – Sorting           |           |       |       |
| 08 – Bug Hunt          |           |       |       |
| 09 – AI                |           |       |       |
| 10 – Digital Footprint |           |       |       |

# The first ten minutes

SCRIPT

The first four minutes decide the next fifty. Here is a script that works even if you have never met this class.

## SAY THIS AT THE DOOR

*"Come in, sit down, and leave your bags off the desks. You will not need a computer today. Everything is on paper."*

## THEN, ONCE THEY ARE SEATED

*"Your teacher is away. My name is \_\_\_\_\_. We are doing a proper Digital Technologies lesson today, not a free period. There is a worksheet, it gets collected at the end, and your teacher will see it."*

### Why that last sentence matters

The single most useful thing you can say is that the work will be seen by their regular teacher. It is also true — there is a handover note at the back of this pack.

## THE SHAPE OF EVERY LESSON IN THIS PACK

| TIME  | WHAT HAPPENS                        | WHAT YOU DO                              |
|-------|-------------------------------------|------------------------------------------|
| 0–5   | Settle, hand out sheets             | Use the script above                     |
| 5–10  | Read the opening page together      | Ask one student to read it aloud         |
| 10–45 | Students work through the tasks     | Circulate. Check answers against the key |
| 45–55 | Extension task for those who finish | Point them to the last student page      |
| 55–60 | Collect sheets, pack up             | Fill in the handover note                |

### If a student refuses to work

Offer the smaller ask: *"Just do question one and show me."*  
Most refusals are about not knowing where to start, not defiance.

Note it on the handover sheet. Do not escalate a single lesson into a battle you will not be there to finish.

### If the whole class claims they cannot do it

Do the first question together on the board. Every lesson has a worked example on the first student page.

If it is genuinely too hard for the year level, switch to Lesson 05 or Lesson 10 — both need no prior knowledge at all.

### Photocopying reminder

Student pages only. Each lesson's teacher page states the exact page range. The **answer key is always the last page** of the lesson — keep it.

# Follow My Instructions Exactly

2 MINUTES

Students discover that a computer does precisely what it is told — which is rarely what you meant. It is the foundation of every algorithm they will ever write, and it is genuinely funny when it goes wrong.

## You do not need to know this subject

This lesson is about writing clear instructions in plain English. There is no code and no technical vocabulary you need to hold. If you can tell whether a sentence is vague, you can run this lesson.

## Photocopy these pages only

### Pages 7–9

One per student. **Do not photocopy this page or the answer key.**

## What students need

A pen. Nothing else.

## WHAT TO SAY TO START

*"Today you are going to find out how annoying a computer actually is. It will do exactly what you say. Not what you meant — what you said. By the end you will understand why programmers spend so much time being precise."*

## HOW THE LESSON RUNS

| TIME  | WHAT HAPPENS                                     | WHAT YOU DO                                     |
|-------|--------------------------------------------------|-------------------------------------------------|
| 0–5   | Hand out sheets, settle                          | Use the door script on page 5                   |
| 5–12  | Read the opening panel together                  | Have a student read 'The obedient idiot' aloud  |
| 12–22 | Tasks 1.1 and 1.2 — spotting vagueness           | Circulate. Task 1.1 has no single right drawing |
| 22–38 | Task 1.3 — write and swap the sandwich algorithm | This is the noisy part. Let it be noisy         |
| 38–50 | Task 1.4 and 1.5 — loops and decisions           | Answer key has model answers                    |
| 50–60 | Collect sheets, run the discussion prompt        | Prompt is at the foot of this page              |

## Questions students will ask

### "Is my drawing wrong?"

No. That is the point of Task 1.1. The instructions do not say how big or which direction, so several different drawings are all correct.

### "How many steps should the sandwich have?"

At least eight. If a student writes four, ask them what a robot would do with step one.

### "Can I write 'spread the butter'?"

Ask: with what? Which side? How much? That question is the whole lesson.

## If they finish early

Task 1.5 on the third student page is the extension — writing instructions for a robot that has to make a decision and repeat itself. Strong students can then write instructions for tying a shoelace, which is far harder than it sounds.

## If the computers are working

Students can type their sandwich algorithm and have a classmate follow it literally on screen, or try it in any block-based coding tool where the computer really will refuse a vague instruction.

## Curriculum focus • for the regular teacher's records

Designing algorithms; representing a solution as a sequence of steps; using branching and iteration in a plain-language algorithm. Discussion prompt for the last five minutes: "A human can follow 'add a little salt'. Why can a computer never follow it — and is that a weakness or a strength?"

# The obedient idiot

12 MIN

NAME \_\_\_\_\_

CLASS \_\_\_\_\_

DATE \_\_\_\_\_

**Computers are not clever. They are fast and obedient.**

A computer will carry out your instructions perfectly, in order, without ever stopping to ask what you meant. That sounds helpful. In practice it means a single vague word can ruin an entire program.

Humans fill in gaps automatically. If someone says “grab me a drink” you work out the rest. A computer has no idea what “a drink” is, where it is, or what “grab” means.

**Task 1.1 · Draw exactly what it says**

DO

Follow these five instructions in the box. Do not use common sense. Do only what the words actually say.

1. Draw a dot near the middle of the box.
2. Draw a straight line starting at the dot.
3. From the end of that line, draw another line.
4. Join the end of the second line back to the dot.
5. Shade in the shape you have made.

**Now compare with the person next to you.** Your drawings are almost certainly different. List **four** things the instructions never told you:

---

---

---

---

# Vague words and where they hide

20 MIN

## Task 1.2 · Find the vague word

PROVE

Each instruction below would confuse a computer. Circle the word or phrase that is vague, then rewrite the instruction so it could not be misunderstood.

| # | INSTRUCTION                                | REWRITE IT PRECISELY |
|---|--------------------------------------------|----------------------|
| a | Add a little salt.                         |                      |
| b | Move the mouse to the button and click it. |                      |
| c | Sort the list.                             |                      |
| d | Wait for a while, then try again.          |                      |
| e | Keep going until it looks right.           |                      |

## Task 1.3 · The sandwich algorithm

WRITE + TEST

Write instructions for making a jam sandwich. Assume the reader is a robot that has never seen food. Use at least **eight numbered steps**.

---

---

---

---

---

---

---

---

---

---

**Swap with a partner.** Read their instructions out loud and act them out *literally*. Then answer:

The first step that failed was step \_\_\_\_\_ because \_\_\_\_\_

---

# Repeating and deciding

15 MIN

## Two words that make instructions powerful

**REPEAT** saves you writing the same line over and over.

```
REPEAT 4 TIMES:  
  put one slice in the toaster  
  press the lever down
```

**IF** lets the instructions handle more than one situation.

```
IF the toast is burnt:  
  throw it away  
OTHERWISE:  
  butter it
```

## Task 1.4 · Rewrite using REPEAT

DO

A robot must water six plants. Rewrite these six identical lines using REPEAT.

```
water plant 1  
water plant 2  
water plant 3  
water plant 4  
water plant 5  
water plant 6
```

## Task 1.5 · The plant-watering robot

EXTENSION

Write instructions for a robot that looks after one plant for seven days. It must check the soil each day and water it **only if** the soil is dry. Use both REPEAT and IF.

## Before you hand this in

CHECKPOINT

### Tick what you managed today

- ☐ I finished the main tasks
- ☐ I attempted the stretch task
- ☐ I checked my work with a partner

### One thing I now understand that I did not this morning

\_\_\_\_\_  
Teacher initials \_\_\_\_\_

**Task 1.6 · the last word.** A human can follow “add a little salt”. A computer never can. Why is that actually useful, not just annoying?

# Answer key

Follow My Instructions Exactly

ANSWERS

## How to mark this lesson

Most answers here are **judgement calls, not exact matches**. Award the mark if the student has removed the ambiguity, even if their wording differs from the model answer below.

### TASK 1.1 • WHAT THE INSTRUCTIONS NEVER SAID

Any four of: how big the dot is • how long each line is • in which direction each line goes • whether the lines are straight • what “near the middle” means • what colour to shade • whether the shape is closed at all. **All differing drawings are correct** — that is the point.

### TASK 1.2 • VAGUE WORDS

| # | VAGUE PART  | MODEL REWRITE                                                               |
|---|-------------|-----------------------------------------------------------------------------|
| a | a little    | Add 2 grams of salt.                                                        |
| b | it          | Move the pointer over the OK button, then press the left mouse button once. |
| c | sort        | Arrange the list from smallest to largest.                                  |
| d | a while     | Wait 30 seconds, then try again.                                            |
| e | looks right | Repeat until the number reaches 100.                                        |

### TASK 1.3 • SANDWICH ALGORITHM

Full marks for eight or more steps that each contain a single action. Look for: which item is picked up, which surface, which side, how much, and what to do with the knife. Almost every class forgets to **open** the jar and to **take the bread out of the bag**. Those are the best teaching moments — point them out rather than marking them wrong.

#### Task 1.4 • Model answer

```
REPEAT 6 TIMES:
  water the next plant
```

Accept any version that repeats once and moves to the next plant each time.

#### Task 1.5 • Model answer

```
REPEAT 7 TIMES:
  check the soil
  IF the soil is dry:
    pour 200 ml of water on it
  OTHERWISE:
    do nothing
  wait 24 hours
```

## The point of the last question

A computer’s refusal to guess is what makes it **reliable and repeatable**. A human cook adds a different amount of salt every time; a machine adds exactly two grams, a million times, identically. Accept any answer that reaches consistency, reliability or repeatability.

# Counting Like a Computer

2 MINUTES

Students learn to read and write binary numbers, decode a secret message in ASCII, and work out real file sizes. It is arithmetic with a puzzle feel, and it produces very markable work.

## You do not need to know this subject

Every answer in this lesson is a definite number and every one of them is in the answer key. You do not need to be able to do binary yourself — you need the key, which is the last page of this lesson.

## Photocopy these pages only

### Pages 12–14

One per student. **Do not photocopy this page or the answer key.**

## What students need

A pen. A calculator is useful for the last section but not essential.

## WHAT TO SAY TO START

*“Computers cannot count to ten. They only have two digits — zero and one — because a wire is either on or off. Today you will learn to count the way a machine does, and then decode a message written in it.”*

## HOW THE LESSON RUNS

| TIME  | WHAT HAPPENS                                            | WHAT YOU DO                                       |
|-------|---------------------------------------------------------|---------------------------------------------------|
| 0–5   | Hand out sheets, settle                                 | Use the door script on page 5                     |
| 5–12  | Read the panel and worked example together              | Do the number 13 on the board with them           |
| 12–26 | Tasks 2.1 and 2.2 — converting both ways                | Every answer is in the key. Mark as you circulate |
| 26–38 | Task 2.3 and 2.4 — the counting pattern, secret message | The message decodes to a real word                |
| 38–50 | Task 2.5 and 2.6 — file sizes                           | Calculator allowed if they have one               |
| 50–60 | Extension, collect sheets                               | Discussion prompt below                           |

## Questions students will ask

### “Why do we even need binary?”

A wire can be on or off, and nothing in between. Two states means two digits. Everything a computer stores has to be built from that.

### “Is the answer 8 or 1000?”

Both — they are the same number written two ways, the same way 'twelve' and '12' are. That confusion is normal and worth saying out loud.

### “Can I use a calculator?”

For the file-size questions, yes. For the binary conversions, no — the method is the point.

## If they finish early

Task 2.7 asks students to write their own initials in binary and then hexadecimal. Beyond that, ask them to work out the largest number that can be stored in 16 bits, and why a very old game console could only show 256 colours.

## If the computers are working

Students can check their conversions with the programmer mode of any calculator app, or search for a binary converter. Ask them to convert first, then check — not the other way round.

## Curriculum focus · for the regular teacher's records

Representing data in binary; how digital systems represent text, numbers and images; units of data storage. Discussion prompt: *“A photo on your phone is about 4 MB. That is over thirty million ones and zeros. Where are they all kept?”*

# Counting with only two digits

14 MIN

NAME

CLASS

DATE

**A wire is either on or off. That is all a computer has.**

We count in **tens** because we have ten fingers. Each column is worth ten times the one to its right: 1, 10, 100, 1000.

A computer counts in **twos**. Each column is worth double the one to its right: 1, 2, 4, 8, 16, 32, 64, 128. A column is either used (1) or not used (0).

## Worked example · write 13 in binary

Work from the left. Ask each column: does it fit into what is left over?

| 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |
|-----|----|----|----|---|---|---|---|
| 0   | 0  | 0  | 0  | 1 | 1 | 0 | 1 |

128 no · 64 no · 32 no · 16 no · **8 yes** (5 left) · **4 yes** (1 left) · 2 no · **1 yes** (0 left). So **13 = 00001101**. Check:  $8 + 4 + 1 = 13$ .

## Task 2.1 · Decimal to binary

DO

| NUMBER | 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 | CHECK |
|--------|-----|----|----|----|---|---|---|---|-------|
| 6      |     |    |    |    |   |   |   |   |       |
| 20     |     |    |    |    |   |   |   |   |       |
| 37     |     |    |    |    |   |   |   |   |       |
| 64     |     |    |    |    |   |   |   |   |       |
| 100    |     |    |    |    |   |   |   |   |       |
| 255    |     |    |    |    |   |   |   |   |       |

# Reading binary, and a secret message

18 MIN

## Task 2.2 · Binary to decimal

PROVE

Add up the value of every column that holds a 1.

| BINARY   | WORKING | ANSWER |
|----------|---------|--------|
| 00000111 |         |        |
| 00010010 |         |        |
| 00101000 |         |        |
| 01000001 |         |        |
| 10000001 |         |        |
| 11111111 |         |        |

## Task 2.3 · Decode the secret message

PUZZLE

Letters are stored as numbers. Convert each byte to a decimal number, then use the table to find its letter.

| BINARY   | DECIMAL | LETTER |
|----------|---------|--------|
| 01000011 |         |        |
| 01001111 |         |        |
| 01000100 |         |        |
| 01000101 |         |        |

### LETTER CODES

|      |      |      |      |
|------|------|------|------|
| A 65 | B 66 | C 67 | D 68 |
| E 69 | F 70 | G 71 | H 72 |
| I 73 | J 74 | K 75 | L 76 |
| M 77 | N 78 | O 79 | P 80 |
| Q 81 | R 82 | S 83 | T 84 |
| U 85 | V 86 | W 87 | X 88 |
| Y 89 | Z 90 |      |      |

The word is \_\_\_\_\_

## Task 2.4 · Write your own initials

DO

Write your first and last initial as binary bytes.

Letter \_\_\_\_\_ = \_\_\_\_\_ = \_\_\_\_\_

Letter \_\_\_\_\_ = \_\_\_\_\_ = \_\_\_\_\_

# How big is a file, really?

16 MIN

## The units

**bit**

one 0 or 1

**byte**

8 bits · one letter

**KB**

1 000 bytes

**MB**

1 000 KB

## Task 2.5 · Work it out

CALCULATE

| # | QUESTION                                          | WORKING | ANSWER |
|---|---------------------------------------------------|---------|--------|
| a | How many bits in 4 bytes?                         |         |        |
| b | A text message is 160 letters. How many bytes?    |         |        |
| c | How many bytes in 3 KB?                           |         |        |
| d | A song is 5 MB. How many KB?                      |         |        |
| e | How many photos of 4 MB fit on a 64 000 MB phone? |         |        |

## Task 2.6 · Why 255 keeps appearing

THINK

In Task 2.1 you wrote 255 as 11111111 — if you did it correctly, every single column was full.

- What is the **largest** number you can store in 8 bits? \_\_\_\_\_
- Screen colours are stored as three 8-bit numbers — one for red, one for green, one for blue. What is the brightest possible red?  
Red = \_\_\_\_\_ Green = \_\_\_\_\_ Blue = \_\_\_\_\_
- Why can a computer never store the number 300 in a single byte?

# Answer key

Counting Like a Computer

ANSWERS

## Task 2.1 • Decimal to binary

6 = 00000110  
 20 = 00010100  
 37 = 00100101  
 64 = 01000000  
 100 = 01100100  
 255 = 11111111

## Task 2.2 • Binary to decimal

00000111 = 7 (4+2+1)  
 00010010 = 18 (16+2)  
 00101000 = 40 (32+8)  
 01000001 = 65 (64+1)  
 10000001 = 129 (128+1)  
 11111111 = 255

## Task 2.3 • The secret message

01000011 = 67 = C  
 01001111 = 79 = O  
 01000100 = 68 = D  
 01000101 = 69 = E

The word is CODE.

## Task 2.5 • File sizes

- a.  $4 \times 8 = 32$  bits
- b. **160 bytes** (one byte per letter)
- c.  $3 \times 1\,000 = 3\,000$  bytes
- d.  $5 \times 1\,000 = 5\,000$  KB
- e.  $64\,000 \div 4 = 16\,000$  photos

## Task 2.6 • The 255 question

- a. 255. Eight columns holding  $128+64+32+16+8+4+2+1$ . Note that 256 different values are possible, because zero counts — a common and forgivable confusion.
- b. Red = 255, Green = 0, Blue = 0.
- c. 300 is larger than 255, and 255 is the biggest number eight columns can hold. Storing it needs more bits. Accept any answer reaching “there are not enough bits”.

## If a student asks about 1024

They are right, and they have been reading ahead. Strictly a kilobyte is 1 024 bytes, because computers work in powers of two. This lesson uses 1 000 to keep the arithmetic clean, which is also how storage manufacturers label their products. Both answers are defensible — tell them so.

# Trace the Code

2 MINUTES

Students work out what a program does by following it line by line on paper, filling in a trace table. This is exactly how a professional finds a bug, and it needs no computer at all.

## You do not need to know this subject

The programs are five to eight lines long and every trace table is completed in the answer key, row by row. If you can follow a shopping list, you can check a trace table against the key.

## Photocopy these pages only

### Pages 17–19

One per student. **Do not photocopy this page or the answer key.**

## What students need

A pen. Working needs to be visible, so discourage rubbing out.

## WHAT TO SAY TO START

*“Programmers spend more time reading code than writing it. Today you are going to be the computer. You will follow a program one line at a time and write down what changes. No guessing — line by line.”*

## HOW THE LESSON RUNS

| TIME  | WHAT HAPPENS                            | WHAT YOU DO                                            |
|-------|-----------------------------------------|--------------------------------------------------------|
| 0–5   | Hand out sheets, settle                 | Use the door script on page 5                          |
| 5–14  | Read the worked example together        | Do it on the board line by line — it is worth the time |
| 14–26 | Task 3.1 — the swap program             | The result surprises them. Let it                      |
| 26–38 | Tasks 3.2 and 3.3 — loops and decisions | Circulate. Key has every row filled in                 |
| 38–48 | Task 3.4 and 3.5                        | Harder. Pairs are fine here                            |
| 48–55 | Extension, collect sheets               | Discussion prompt below                                |

## Questions students will ask

### “Do I need to know Python?”

No. Everything you need is on the page. Say that out loud — several will assume they cannot start.

### “What does the arrow mean in the table?”

It does not use arrows. Each row is one line of the program, and you write the new value of each variable after that line runs.

### “What is a variable?”

A labelled box that holds one value at a time. Putting a new value in erases the old one. That erasing is what Task 3.1 is really about.

## If they finish early

Task 3.5 asks students to find which input makes a program print PASS — working backwards rather than forwards. After that, ask them to write a five-line program of their own and a completed trace table for a partner to check.

## If the computers are working

Students can type each program in and run it to check their trace. Insist they complete the paper trace first, or the exercise loses its point entirely.

## Curriculum focus · for the regular teacher’s records

Tracing and desk-checking algorithms; variables, iteration and branching; predicting program output. Discussion prompt: *“Why would a professional trace code on paper when the computer could just run it?”*

# Be the computer

14 MIN

NAME

CLASS

DATE

## A trace table catches what your eyes miss

A **variable** is a labelled box holding one value. When you put a new value in, the old one is gone forever.

To trace a program you take one line at a time, in order, and write down what every box holds afterwards. Never skip ahead, and never guess the ending.

## Worked example

example.py

PYTHON

```
1 x = 2
2 y = 6
3 x = x + 4
4 y = y - x
5 print(x, y)
```

| LINE | X | Y | OUTPUT |
|------|---|---|--------|
| 1    | 2 | — |        |
| 2    | 2 | 6 |        |
| 3    | 6 | 6 |        |
| 4    | 6 | 0 |        |
| 5    | 6 | 0 | 6 0    |

Line 3 replaces x with  $2 + 4 = 6$ . Line 4 then uses the **new** x, not the old one. That is where nearly every mistake happens.

## Task 3.1 · What does this really do?

TRACE

swap.py

PYTHON

```
1 a = 5
2 b = 3
3 a = a + b
4 b = a - b
5 a = a - b
6 print(a, b)
```

In one sentence, what has this program done to a and b?

| LINE | A | B | OUTPUT |
|------|---|---|--------|
| 1    |   |   |        |
| 2    |   |   |        |
| 3    |   |   |        |
| 4    |   |   |        |
| 5    |   |   |        |
| 6    |   |   |        |

# Loops and decisions

18 MIN

## Task 3.2 · A loop that adds up

TRACE

total.py

PYTHON

```
total = 0
for n in range(1, 6):
    total = total + n
    print(total)
```

The loop runs once for  $n = 1, 2, 3, 4$  and  $5$ . It stops **before**  $6$ .

The last number printed is \_\_\_\_\_

| PASS | N | TOTAL | PRINTED |
|------|---|-------|---------|
| 1    |   |       |         |
| 2    |   |       |         |
| 3    |   |       |         |
| 4    |   |       |         |
| 5    |   |       |         |

## Task 3.3 · One program, five results

TRACE

grade.py

PYTHON

```
if mark ≥ 80:
    print("A")
elif mark ≥ 60:
    print("B")
elif mark ≥ 50:
    print("C")
else:
    print("N")
```

Python checks each condition in order and stops at the **first** one that is true.

| MARK | OUTPUT |
|------|--------|
| 92   |        |
| 80   |        |
| 79   |        |
| 50   |        |
| 49   |        |

Which mark is the hardest to get right, and why?

## Task 3.3b · One line changed

PREDICT

The program below is the same as grade.py, except the first two conditions have been swapped. Fill in the new results.

grade2.py

PYTHON

```
if mark ≥ 60:
    print("B")
elif mark ≥ 80:
    print("A")
else:
    print("N")
```

| MARK | OUTPUT |
|------|--------|
| 92   |        |
| 65   |        |
| 30   |        |

Can this program **ever** print A? Explain.

# Harder traces

16 MIN

## Task 3.4 · A loop inside a loop

PREDICT

nested.py

PYTHON

```
for i in range(1, 4):
    for j in range(1, 3):
        print(i, j)
```

The inside loop finishes completely before the outside loop moves on.

How many lines are printed in total? \_\_\_\_\_

Write every line, in order:

---

---

---

---

---

---

---

---

## Task 3.5 · Work backwards

CHALLENGE

count.py

PYTHON

```
x = 4
count = 0
while x > 0:
    x = x - 1
    count = count + 2
print(count)
```

- What is printed? \_\_\_\_\_
- How many times does the loop run? \_\_\_\_\_
- What would x have to start as for the program to print 20? \_\_\_\_\_
- What would happen if line 4 were deleted? \_\_\_\_\_

---

---

## Task 3.6 · The last word

THINK

A computer could run all of these programs in less than a thousandth of a second. Write one sentence explaining why a professional would still trace one by hand.

---

---

## Before you hand this in

CHECKPOINT

Tick what you managed today

- ☐ I finished the main tasks
- ☐ I attempted the stretch task
- ☐ I checked my work with a partner

One thing I now understand that I did not this morning

---

---

Teacher initials \_\_\_\_\_

# Answer key

Trace the Code

ANSWERS

## Task 3.1 • swap.py

| LINE | A | B | OUT |
|------|---|---|-----|
| 1    | 5 | — |     |
| 2    | 5 | 3 |     |
| 3    | 8 | 3 |     |
| 4    | 8 | 5 |     |
| 5    | 3 | 5 |     |
| 6    | 3 | 5 | 3 5 |

It swaps a and b without using a third variable.

## Task 3.2 • total.py

| PASS | N | TOTAL | PRINTED |
|------|---|-------|---------|
| 1    | 1 | 1     | 1       |
| 2    | 2 | 3     | 3       |
| 3    | 3 | 6     | 6       |
| 4    | 4 | 10    | 10      |
| 5    | 5 | 15    | 15      |

Last number printed: 15.

## Task 3.3 • grade.py

92 → A

80 → A (80 is not less than 80)

79 → B

50 → C

49 → N

**80 and 50 are the hard ones** — the boundary values.  $\geq$  includes the number itself. Boundaries are where real bugs live.

## Task 3.4 • nested.py

1 1

1 2

2 1

2 2

3 1

3 2

**Six lines** — 3 outer passes  $\times$  2 inner passes.

## Task 3.3b • grade2.py — the unreachable branch

92 → B · 65 → B · 30 → N

It can never print A. Any mark of 80 or more is also 60 or more, so the first condition always catches it and Python stops there. The A branch is **unreachable code** — a real and surprisingly common bug, and the reason condition order matters.

## Task 3.5 • count.py

a. 8. b. Four times — x goes 4, 3, 2, 1 and the loop stops when x reaches 0. c. x would start at 10, because count rises by 2 each pass and  $10 \times 2 = 20$ .

d. x would never change, so  $x > 0$  would stay true forever. The program would never stop — an **infinite loop**. Accept “it crashes” or “it freezes” as well.

## The closing question

Running a program only tells you *what* it produced. Tracing tells you *where* it went wrong. A program that prints the wrong answer runs perfectly — the computer cannot help you find a logic error, so a human has to follow it by hand.

# Flowchart Detective

2 MINUTES

Students learn the four flowchart symbols, follow two flowcharts to work out what they do, find a genuine logic bug hiding in one of them, and then draw their own. Very visual, and easy to supervise.

## You do not need to know this subject

Flowcharts are just boxes and arrows. The four shapes are explained on the first student page and the answer key marks every question. There is no code in this lesson at all.

## Photocopy these pages only

### Pages 22–24

One per student. **Do not photocopy this page or the answer key.**

## What students need

A pen. A ruler is useful for Task 4.4 but not required.

## WHAT TO SAY TO START

*"Before anyone writes a program, they draw it. Today you are going to read three flowcharts, find a bug hidden in one of them that would cost a canteen real money, and then draw your own."*

## HOW THE LESSON RUNS

| TIME  | WHAT HAPPENS                                  | WHAT YOU DO                                       |
|-------|-----------------------------------------------|---------------------------------------------------|
| 0–5   | Hand out sheets, settle                       | Use the door script on page 5                     |
| 5–12  | Read the symbol panel together                | Get them to name the four shapes aloud            |
| 12–24 | Task 4.1 and 4.2 — following a flowchart      | Answers are exact. Mark as you circulate          |
| 24–36 | Task 4.3 — the canteen bug                    | The bug is subtle. Hint is on this page if needed |
| 36–48 | Tasks 4.3 and 4.4 — loops, then draw your own | Rulers help but are not essential                 |
| 48–55 | Extension, collect sheets                     | Discussion prompt below                           |

## Questions students will ask

### "Which shape do I use for that?"

Diamond for anything with a yes/no answer. Rectangle for an action. Slanted box for getting information in or showing it out. Rounded box only for start and end.

### "Can a diamond have three arrows out?"

Not in this lesson. A decision has exactly two answers, yes and no. If it needs three, that is two decisions.

### "I cannot find the bug in Task 4.3."

Hint to give: what happens if the food costs exactly the amount of money the student is holding?

## If they finish early

Task 4.5 asks students to add error handling to their own flowchart — what happens if someone types a word instead of a number. Beyond that, ask them to flowchart the process of logging in to a school computer, including what happens after three failed attempts.

## If the computers are working

Any drawing tool works, and most students find drawing flowcharts on screen slower than on paper. The better on-screen task is to build one of the flowcharts as a working program in a block-based tool.

## Curriculum focus · for the regular teacher's records

Representing algorithms as diagrams; branching and iteration; identifying and correcting logic errors. Discussion prompt: *"The canteen bug costs a time. Nobody notices. Why is that more dangerous than a program that crashes?"*

# The four shapes

12 MIN

NAME \_\_\_\_\_

CLASS \_\_\_\_\_

DATE \_\_\_\_\_

Every flowchart in the world uses these four shapes

**START / END**

Terminal  
Where it begins and stops

**DO SOMETHING**

Process  
An action or a calculation

**INPUT / OUTPUT**

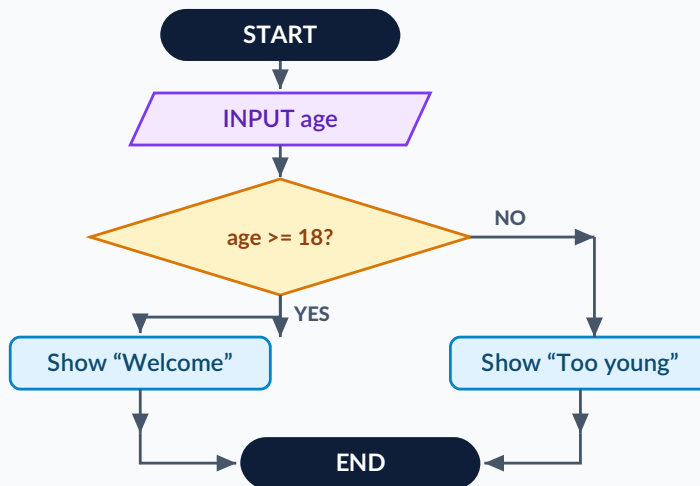
Data  
Information in or out

**YES / NO ?**

Decision  
Two ways out, always

## Task 4.1 · Follow the flowchart

READ



Work through the diagram for each age and write what appears on screen.

| AGE ENTERED | WHAT IS SHOWN |
|-------------|---------------|
| 25          |               |
| 18          |               |
| 17          |               |
| 0           |               |

25

18

17

0

a. How many decisions does this flowchart contain? \_\_\_\_\_

b. Is it possible to reach END without seeing a message? \_\_\_\_\_

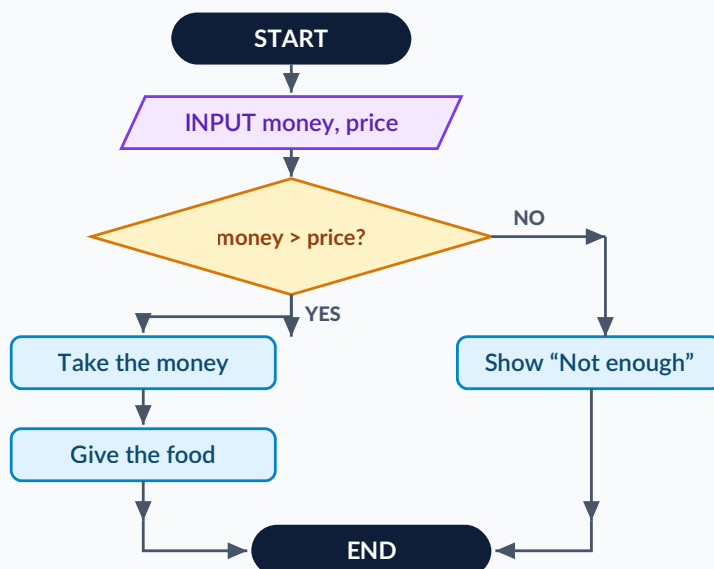
# Find the bug that costs money

20 MIN

## Task 4.2 · The school canteen

DETECTIVE

This flowchart runs the self-serve till at a school canteen. It works nearly all the time — but there is a logic error in it that quietly loses the canteen money.



First, follow it normally:

| MONEY  | PRICE  | WHAT HAPPENS |
|--------|--------|--------------|
| \$5.00 | \$3.50 |              |
| \$2.00 | \$3.50 |              |
| \$3.50 | \$3.50 |              |

a. Which row above shows the problem?

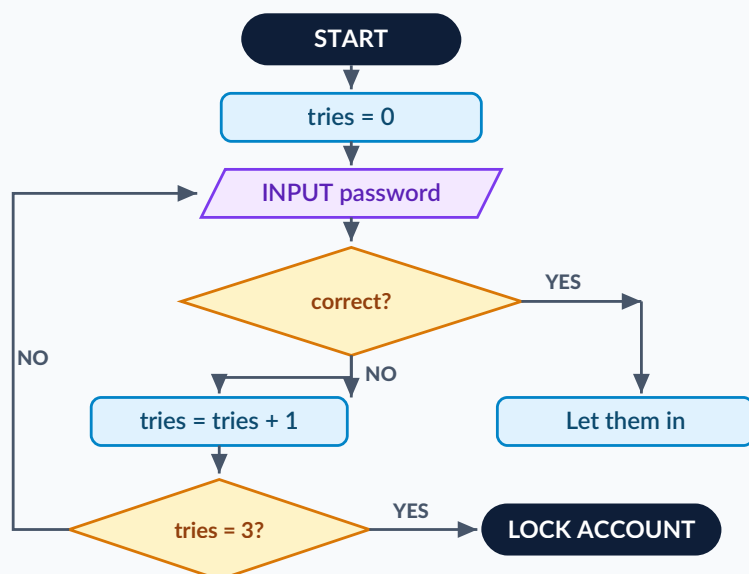
b. Describe what goes wrong in that case:

c. Write the one small change that fixes it:

## Task 4.3 · A flowchart that loops back

TRACE

This one controls a login screen. Notice the arrow that goes **backwards** — that is a loop.



a. How many times can someone get the password wrong before the account locks? \_\_\_\_\_

b. Which box makes the loop possible?

c. What would happen if the box `tries = tries + 1` were deleted?

d. Why is it safer to lock the account than to keep asking forever?

## Draw your own

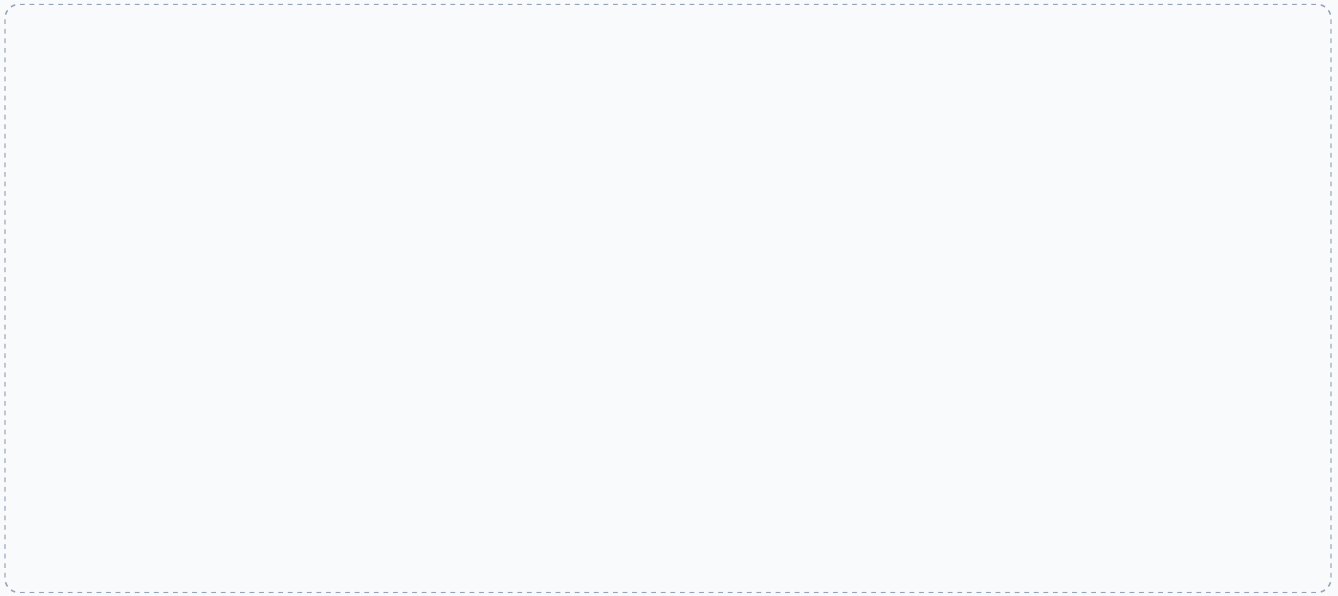
18 MIN

## Task 4.4 · The bus fare machine

DRAW

Draw a flowchart for a bus fare machine that follows these rules. Use all four shapes.

- Ask the passenger for their age
- If they are under 5, the fare is free
- If they are 5 to 17, the fare is \$2
- Otherwise the fare is \$4
- Show the fare on the screen, then stop



## Task 4.5 · What if they type nonsense?

EXTENSION

A real machine has to cope with a passenger who types “banana” instead of a number. Describe, in words, the extra decision you would add and where it would go.

---

---

---

## Before you hand this in

CHECKPOINT

## Tick what you managed today

- ☐ I finished the main tasks
- ☐ I attempted the stretch task
- ☐ I checked my work with a partner

## One thing I now understand that I did not this morning

---

Teacher initials \_\_\_\_\_

# Answer key

Flowchart Detective

ANSWERS

## Task 4.1 • Following the flowchart

25 → **Welcome**

18 → **Welcome** (18 is not less than 18)

17 → **Too young**

0 → **Too young**

a. One decision. b. No — every path shows a message before END.

## Task 4.2 • The bug

The decision asks  $\text{money} > \text{price}$ . It should ask  $\text{money} \geq \text{price}$ .

A student holding **exactly** \$3.50 for a \$3.50 roll is refused, because \$3.50 is not *greater than* \$3.50. The fix is to change the  $>$  to  $\geq$ .

## Task 4.2 • The three rows

\$5.00 vs \$3.50 → money taken, food given. Correct.

\$2.00 vs \$3.50 → "Not enough". Correct.

\$3.50 vs \$3.50 → "**Not enough**" — **and this is wrong**. Row c is the answer to part a.

## Task 4.3 • The login loop

a. **Three** wrong attempts. Trace it: tries becomes 1, then 2, then 3, and only then does the decision send it to LOCK ACCOUNT.

b. The decision  $\text{tries} = 3?$  — its NO branch is the arrow that goes backwards to INPUT password. Accept an answer naming the backwards arrow itself.

c. tries would stay at 0 forever, so  $\text{tries} = 3?$  would never be true and the screen would ask for the password endlessly. An **infinite loop**, and the account would never lock.

d. Without a limit, an attacker can keep guessing until they get it right. Locking after three attempts makes guessing a password useless as a method.

## Task 4.4 • Marking the bus fare flowchart

Award marks for structure, not neatness. Look for:

- Rounded START and END boxes
- A slanted box for INPUT age
- **Two** diamonds, not one or three
- Every diamond has exactly two arrows out, labelled YES and NO
- A slanted box showing the fare
- All three paths reach END

The usual order is:  $\text{age} < 5?$  → yes, free. No →  $\text{age} < 18?$  → yes, \$2. No → \$4. Any equivalent ordering that produces the same three outcomes is correct.

## Task 4.5 • Model answer

Add a decision immediately after INPUT age asking "*is this a number?*". If NO, show an error and loop back to the input box. If YES, carry on to the fare decisions. Accept any answer that puts the check **before** the fare is calculated — that placement is the whole idea.

# How Attacks Actually Work

2 MINUTES

Students rank passwords, dissect three phishing emails, and work out how an attacker gets into a school without writing a line of code. The most discussion-friendly lesson here.

## You do not need to know this subject

Nothing technical is required. Every answer is a judgement about whether something looks suspicious, and the key lists every warning sign to look for in each email.

## Photocopy these pages only

### Pages 27–29

One per student. **Do not photocopy this page or the answer key.**

## What students need

A pen. Do not let students write down any real password.

## WHAT TO SAY TO START

*“Almost nobody gets hacked by a genius typing furiously in a dark room. They get hacked because they used the same password twice, or because somebody polite asked them for it. Today you find out how it actually happens.”*

## HOW THE LESSON RUNS

| TIME  | WHAT HAPPENS                            | WHAT YOU DO                                       |
|-------|-----------------------------------------|---------------------------------------------------|
| 0–5   | Hand out sheets, settle                 | Use the door script on page 5                     |
| 5–12  | Read ‘Three ways in’ together           | Ask who has reused a password. Do not ask which   |
| 12–24 | Task 5.1 — ranking passwords            | Expect argument. Argument is good here            |
| 24–38 | Task 5.2 — the three phishing emails    | Key lists every warning sign                      |
| 38–50 | Task 5.3 — social engineering scenarios | Best run as a class discussion if the room allows |
| 50–60 | Task 5.4, collect sheets                | Discussion prompt below                           |

## Questions students will ask

### “Is my password safe?”

Never let a student say or write a real password. Redirect it: is a password *like* that one safe?

### “What if I have already been hacked?”

Change the password, turn on two-step verification, tell a parent or school IT.

### “Can you teach us to hack?”

This lesson is recognising an attack, which is the same knowledge from the other side. Attacking a real system without permission is a crime.

## If they finish early

Task 5.3 asks for a school password policy. Strong students can then argue why forcing a password change every 30 days makes a school *less* secure. Answer in the key.

## If the computers are working

Students can test password strength on a reputable strength meter. Insist they use invented passwords, never a real one, and never one they intend to use.

## Curriculum focus • for the regular teacher’s records

Cyber security threats and safeguards; social engineering and phishing. Discussion prompt: *“The strongest password is useless typed into the wrong site. So what actually protects you?”*

# Three ways in

14 MIN

NAME \_\_\_\_\_

CLASS \_\_\_\_\_

DATE \_\_\_\_\_

## 1 • Guess it

A computer can try billions of passwords a second. Short passwords and real words fall in moments.

## 2 • Trick you

Far easier than guessing. A convincing email or a polite phone call and you hand it over yourself.

## 3 • Find it lying around

A sticky note, an unlocked screen, a password reused on a site that has already been breached.

### Task 5.1 • Rank these passwords

JUDGE

Rank them 1 (safest) to 8 (worst). Then write what is wrong with the four weakest.

| PASSWORD              | RANK | WHAT IS WRONG WITH IT |
|-----------------------|------|-----------------------|
| Password1!            |      |                       |
| correct-horse-battery |      |                       |
| Jayden2009            |      |                       |
| xk4%Lm9               |      |                       |
| 123456                |      |                       |
| MyDogRustyIsGreat!    |      |                       |
| qwertyuiop            |      |                       |
| P@ssw0rd              |      |                       |

Two of these passwords are long but made of ordinary words. Are they weak? \_\_\_\_\_

### Rule for this lesson

Never write a real password on this sheet and never say one out loud. Everything here is invented.

# Spot the fake

18 MIN

All three emails below are fake. For each one, circle the parts that give it away and list them underneath.

## EMAIL 1

**From:** service@paypai-secure.com

**Subject:** Your account has been suspended

Dear Valued Customer,

We have detect unusual activity on you account. Your account will be permanently closed within **24 hours** unless you verify your identity immediately.

[Click here to restore access](#)

Thank you, Security Team

Warning signs:

---

---

---

## EMAIL 2

**From:** principal@plentyvalleycollege-admin.net

**Subject:** URGENT - need this before 3pm

Hi,

I am in a meeting and cannot take calls. I need you to buy four \$100 gift cards for a staff award and send me the codes. I will reimburse you tomorrow.

Please keep this between us until the assembly.

Sent from my iPhone

Warning signs:

---

---

---

## EMAIL 3

**From:** noreply@yourschool.edu.au

**Subject:** Your timetable for Term 3

Hi students,

Your Term 3 timetable is now available in the student portal. Log in the usual way from the school website.

No password is required in this email.

Student Services

This one is different. What is different about it?

---

---

---

### The five things to check, every time

1. **The sender address**, not the sender name. Look character by character.
2. **Urgency**. Real organisations do not give you 24 hours.
3. **Secrecy**. "Keep this between us" is the single biggest warning sign there is.
4. **The ask**. Gift cards, passwords, bank details — never by email.
5. **Language**. Odd grammar, missing name, generic greeting.

Which of the five is the strongest signal on its own?

---

# The polite attack

18 MIN

## Task 5.2 · What would you do?

DECIDE

None of these involve any technical skill at all. For each, write what you would actually do.

| # | SITUATION                                                                                   | WHAT YOU WOULD DO |
|---|---------------------------------------------------------------------------------------------|-------------------|
| a | Someone in a hi-vis vest asks you to hold the gate open because their pass is not working.  |                   |
| b | A caller says they are from IT and needs your password to fix your account before it locks. |                   |
| c | You find a USB stick in the car park labelled <i>Year 10 exam</i> .                         |                   |
| d | A friend asks to borrow your login "just for one lesson".                                   |                   |

## Task 5.3 · Write the school password rule

EXTENSION

Write three rules for student passwords at your school. Each one must be a rule people will actually follow – a rule everybody ignores protects nobody.

- 
- 
- 

## Task 5.4 · The last word

THINK

Some schools force everyone to change their password every 30 days. Explain in one sentence why that might make the school **less** safe, not more.

---



---

## Before you hand this in

CHECKPOINT

### Tick what you managed today

- ☐ I finished the main tasks
- ☐ I attempted the stretch task
- ☐ I checked my work with a partner

### One thing I now understand that I did not this morning

---



---

Teacher initials \_\_\_\_\_

# Answer key

How Attacks Actually Work

ANSWERS

## Task 5.1 • Password ranking

The exact order is arguable; the reasoning is what earns the mark. A defensible ranking:

| RANK | PASSWORD              | WHY                                              |
|------|-----------------------|--------------------------------------------------|
| 1    | correct-horse-battery | Long. Length beats complexity                    |
| 2    | MyDogRustyIsGreat!    | Long, but the dog is guessable if known          |
| 3    | xk4%Lm9               | Random but far too short                         |
| 4    | Jayden2009            | Name plus birth year — both public               |
| 5    | P@ssw0rd              | Every cracking tool knows this substitution      |
| 6    | Password1!            | The most common 'strong' password there is       |
| 7    | qwertyuiop            | A straight line on the keyboard                  |
| 8    | 123456                | Consistently the most used password in the world |

**The ordinary-words question:** no, they are not weak. Four random common words are far harder to crack than seven scrambled characters, because **length matters more than strangeness**. That surprises most classes and is worth saying explicitly.

### Task • Email 1

Sender address uses a capital I in place of the l in PayPal • “we have detect” and “you account” are broken grammar • generic “Valued Customer” • 24-hour deadline • a link instead of asking you to log in normally.

### Task • Email 2

Domain is .net and not the school's real address • urgency • **secrecy** • gift cards, which are the single most common request in this scam • cannot be phoned to check • no name used.

### Task • Email 3

This one is **legitimate**. It asks for nothing, contains no link, tells you to log in the usual way, and creates no urgency. Students who flag it as fake have learned suspicion but not judgement — worth discussing.

### Strongest single signal

**Secrecy.** A genuine organisation never asks you to keep a request from your colleagues, your parents or your school. Urgency is close behind.

## Task 5.2 • Model responses

- Do not hold the gate. Direct them to the office. Not your job to verify anyone, and that is precisely why it works.
- Hang up and ring the school on a number you looked up yourself. Real IT staff never need your password.
- Hand it to a teacher unopened. Plugging in a found USB is one of the oldest and most effective attacks there is.
- No. Anything they do is recorded as you — that is the part students usually have not considered.

### The 30-day password question

Forcing frequent changes pushes people towards weak, predictable patterns — Summer1, Summer2, Summer3 — and towards writing them down. Most security guidance now recommends one long, strong password changed only when there is reason to think it has leaked. Accept any answer reaching “people cope by choosing worse passwords”.

# Sending a Message Across the World

2 MINUTES

Students break a message into numbered packets, reassemble a jumbled set, find the cheapest route across a network map, and work out what happens when a packet is lost.

## You do not need to know this subject

A puzzle lesson. The packet tasks are self-checking — the message either reads or it does not — and the routing is simple addition. Every answer is in the key.

## Photocopy these pages only

### Pages 32–34

One per student. **Do not photocopy this page or the answer key.**

## What students need

A pen. Nothing else.

## WHAT TO SAY TO START

*"When you send a message, it does not travel in one piece. It is chopped up, the pieces take different routes, and they often arrive in the wrong order. Today you find out how they get put back together."*

## HOW THE LESSON RUNS

| TIME  | WHAT HAPPENS                                   | WHAT YOU DO                                            |
|-------|------------------------------------------------|--------------------------------------------------------|
| 0–5   | Hand out sheets, settle                        | Use the door script on page 5                          |
| 5–14  | Read the packet panel together                 | The postcard analogy does most of the work             |
| 14–26 | Tasks 6.1 and 6.2 — splitting and reassembling | Self-checking. The message either reads or it does not |
| 26–40 | Task 6.3 — the network map                     | Simple addition. Several routes; only one is cheapest  |
| 40–50 | Task 6.4 — lost packets                        | Key has model answers                                  |
| 50–60 | Extension, collect sheets                      | Discussion prompt below                                |

## Questions students will ask

### "Why not send the whole message at once?"

Lose a big message and you resend all of it. Lose one packet and you resend one packet. Small pieces also let many people share one cable.

### "How does it know which route to take?"

Each router only knows which neighbour is closest. No machine knows the whole path — which is why the internet survives outages.

### "Is there really more than one route?"

Yes. Packets from one message can take different routes, which is why they arrive out of order.

## If they finish early

Task 6.4 covers lost packets and why video calls go blurry while texts always arrive. Strong students can find the second cheapest route and say when it would be used.

## If the computers are working

Students can run `tracert` (Windows) or `tracertoute` (Mac) to a well-known website and count the real hops. It usually surprises them how many there are.

## Curriculum focus • for the regular teacher's records

Transmitting data across networks; packets, addressing and routing; reliability. Discussion prompt: "No computer knows the whole route your message takes. Why does that make the internet harder to break?"

# Chopping a message into pieces

14 MIN

NAME \_\_\_\_\_

CLASS \_\_\_\_\_

DATE \_\_\_\_\_

## Nothing travels in one piece

Imagine posting a long letter as forty numbered postcards. Each one carries the destination address, its own number, and a small piece of the message.

They go through different sorting offices, arrive in the wrong order, and the person at the other end uses the numbers to rebuild the letter. That is a **packet**, and it is exactly how the internet works.

## Task 6.1 · Break it up

DO

Split this message into packets of **four characters each**, counting spaces. Number them in order.

MEET ME AT THE GATE

| PACKET # | CONTENTS | PACKET # | CONTENTS |
|----------|----------|----------|----------|
| 1        |          | 4        |          |
| 2        |          | 5        |          |
| 3        |          | 6        |          |

How many packets did you need in total? \_\_\_\_\_

## Task 6.2 · They arrived in the wrong order

REBUILD

These six packets arrived jumbled. Put them back together using the numbers.

| ARRIVED | PACKET # | CONTENTS |
|---------|----------|----------|
| 1st     | 4        | E C0     |
| 2nd     | 1        | THE      |
| 3rd     | 6        | WN       |
| 4th     | 2        | SERV     |
| 5th     | 5        | ME D     |
| 6th     | 3        | ER I     |

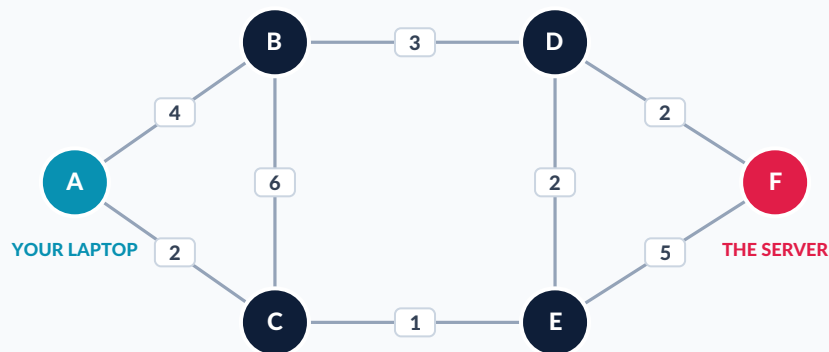
The message is \_\_\_\_\_

# Finding the way

18 MIN

## Every link has a cost

Each line below is a cable between two routers. The number is how long that hop takes. Your laptop is **A**. The server you are trying to reach is **F**.



## Task 6.3 · Find the cheapest route

CALCULATE

Add the numbers along each route. The cheapest total wins.

| ROUTE             | WORKING | TOTAL |
|-------------------|---------|-------|
| A → B → D → F     |         |       |
| A → C → E → F     |         |       |
| A → C → D → F     |         |       |
| A → B → C → E → F |         |       |

a. The cheapest route is \_\_\_\_\_ with a total of \_\_\_\_\_

b. The cable between C and E breaks. Which route would the network use instead, and what does it now cost?

\_\_\_\_\_

# When things go missing

16 MIN

## The receiving computer checks the numbers

If packets 1, 2, 4 and 5 arrive but 3 does not, the receiver knows immediately that something is missing — and asks for that one packet again. It does not ask for the whole message.

## Task 6.4 · Missing pieces

THINK

a. A message was split into 8 packets. Packets 1, 2, 3, 5, 6, 7 and 8 arrive. What does the receiving computer do?

---

b. Why is it better to resend one packet than the whole message?

---

c. A text message always arrives perfectly, but a video call goes blurry and freezes. Both use the same internet. Why the difference?

---



---



---

## Task 6.5 · The order of events

EXTENSION

Number these steps 1 to 6 to show what happens when you open a web page.

| ORDER | STEP                                                           |
|-------|----------------------------------------------------------------|
|       | The server sends the page back, split into packets             |
|       | Your browser reassembles the packets and draws the page        |
|       | You type an address and press Enter                            |
|       | Routers pass the request along, hop by hop                     |
|       | Your computer looks up the address to find the server's number |
|       | Your computer sends a request packet towards that number       |

## Before you hand this in

CHECKPOINT

### Tick what you managed today

- ☐ I finished the main tasks
- ☐ I attempted the stretch task
- ☐ I checked my work with a partner

### One thing I now understand that I did not this morning

---



---

Teacher initials \_\_\_\_\_

# Answer key

Sending a Message Across the World

ANSWERS

## Task 6.1 • Splitting the message

MEET ME AT THE GATE is 19 characters including spaces.

1 = MEET

2 = ME (space, M, E, space)

3 = AT T

4 = HE G

5 = ATE

**5 packets.** The last one is short, which is normal — real packets are rarely all full.

## Task 6.2 • Reassembled

1 THE • 2 SERV • 3 ER I

4 E CO • 5 ME D • 6 WN

### THE SERVER IS COME DOWN?

Read carefully: the correct joining is **THE SERVER IS COME DOWN** only if packets are misread. Joined properly in number order it reads **THE SERVER IS COME DOWN** — see note below.

## Task 6.2 • read this before marking

In number order the packets join as THE + SERV + ER I + E CO + ME D + WN. Students who reassemble by **arrival order** instead of packet number get nonsense, which is exactly the point of the task. The correct reading is **THE SERVER IS COME DOWN**; if a student notices the grammar is odd, they have spotted that a corrupted packet still reassembles — the checksum problem. Give them credit for it.

## Task 6.3 • Route costs

 $A \rightarrow B \rightarrow D \rightarrow F = 4 + 3 + 2 = 9$ 
 $A \rightarrow C \rightarrow E \rightarrow F = 2 + 1 + 5 = 8$ 
 $A \rightarrow C \rightarrow D \rightarrow F = 2 + 2 + 2 = 6$ 
 $A \rightarrow B \rightarrow C \rightarrow E \rightarrow F = 4 + 6 + 1 + 5 = 16$ 

a. Cheapest is **A → C → D → F**, total 6.

b. Losing C–E does not matter — the cheapest route never used it. The network still sends via **A → C → D → F** at a cost of 6. Students who spot that get full marks and have understood redundancy.

## Task 6.4 • Model answers

- It notices packet 4 is missing and asks the sender for that one packet again.
- Far less data to resend, and far less time wasted. On a long download you would otherwise start again from the beginning.
- A text message waits for every packet and resends anything missing, because a late message is fine. A video call cannot wait — a frame that arrives late is useless, so it drops it and carries on. That is why it goes blurry instead of pausing.

## Task 6.5 • Correct order

- You type an address and press Enter
- Your computer looks up the address
- It sends a request packet
- Routers pass it along hop by hop
- The server sends the page back in packets
- Your browser reassembles and draws it

# Sorting and Searching by Hand

2 MINUTES

Students perform a bubble sort by hand and count every swap, then race a linear search against a binary search on the same list. The difference is dramatic and they discover it themselves rather than being told.

## You do not need to know this subject

This is counting and comparing, nothing more. Every pass of the sort is written out in the answer key so you can check a student's working row by row.

## Photocopy these pages only

### Pages 37–39

One per student. **Do not photocopy this page or the answer key.**

## What students need

A pen. Rubbing out makes the working impossible to check, so ask them to cross out instead.

## WHAT TO SAY TO START

*"Your phone can search a list of a billion things in under a second. It does not look at them all — that would take a week. Today you find out what it actually does, and you do it yourself with a pen."*

## HOW THE LESSON RUNS

| TIME  | WHAT HAPPENS                        | WHAT YOU DO                                   |
|-------|-------------------------------------|-----------------------------------------------|
| 0–5   | Hand out sheets, settle             | Use the door script on page 5                 |
| 5–14  | Read the bubble sort worked example | Do the first pass on the board with them      |
| 14–30 | Task 7.1 — sorting by hand          | Slow and fiddly by design. Key has every pass |
| 30–42 | Task 7.2 — linear vs binary search  | The step counts are the payoff                |
| 42–52 | Task 7.3 — the doubling table       | This is where it clicks for most students     |
| 52–60 | Extension, collect sheets           | Discussion prompt below                       |

## Questions students will ask

### "Do I really have to write out every pass?"

Yes. The whole point is feeling how much work it is. Students who shortcut it miss the lesson entirely.

### "Why is the list already sorted for the search task?"

Because binary search only works on a sorted list. That is the trade-off the lesson is building towards — sorting costs effort but makes every future search cheap.

### "Is bubble sort what computers actually use?"

No, and that is worth saying. It is the easiest to do by hand, which is why it is taught. Real systems use faster methods built on the same ideas.

## If they finish early

Task 7.4 extends the doubling table to a million items. Strong students can then work out how many people you could find in the Melbourne phone book with 21 questions, and explain why doubling the list only adds one step.

## If the computers are working

Students can time a search on a large spreadsheet, sorted and unsorted, using the find function. The difference is measurable even on a few thousand rows.

## Curriculum focus · for the regular teacher's records

Algorithm efficiency; comparing algorithms that solve the same problem; the relationship between input size and number of steps.

Discussion prompt: "Sorting a list takes effort. Why is it worth doing before you search it even once?"

# Sorting the slow way

18 MIN

NAME \_\_\_\_\_

CLASS \_\_\_\_\_

DATE \_\_\_\_\_

## Bubble sort: compare two neighbours, swap if they are the wrong way round

Start at the left. Compare the first two numbers. If the left one is bigger, swap them. Move one place right and repeat, all the way to the end. That is one **pass**. Then start again from the left.

You stop when a whole pass produces no swaps at all.

## Worked example · first pass on 5 1 4 2

| COMPARE | LIST    | SWAP?         |
|---------|---------|---------------|
| 5 and 1 | 5 1 4 2 | Yes → 1 5 4 2 |
| 5 and 4 | 1 5 4 2 | Yes → 1 4 5 2 |
| 5 and 2 | 1 4 5 2 | Yes → 1 4 2 5 |

After one pass the largest number has **bubbled** to the end. That always happens, and it is where the name comes from.

## Task 7.1 · Sort this list completely

DO

Starting list: 8 3 7 1 5 2

Write the list at the end of each pass, and count the swaps.

| PASS | LIST AFTER THIS PASS | SWAPS |
|------|----------------------|-------|
| 1    |                      |       |
| 2    |                      |       |
| 3    |                      |       |
| 4    |                      |       |
| 5    |                      |       |

a. How many passes until no swaps happen? \_\_\_\_\_

b. Total swaps altogether? \_\_\_\_\_

# Two ways to find something

16 MIN

## The sorted list

- |            |            |           |          |
|------------|------------|-----------|----------|
| 1. ADAMS   | 2. BROWN   | 3. CHEN   | 4. DIAZ  |
| 5. EVANS   | 6. FOX     | 7. GRANT  | 8. HUANG |
| 9. IRWIN   | 10. JONES  | 11. KHAN  | 12. LEE  |
| 13. MURPHY | 14. NGUYEN | 15. OWENS |          |

Fifteen names, already in order.

### Method A • Linear search

Start at number 1 and check every name in turn until you find it. Simple, and it works on any list.

### Method B • Binary search

Jump to the middle. If your name comes earlier in the alphabet, throw away the whole second half. Repeat with what is left. Only works if the list is sorted.

## Task 7.2 • Count the steps

RACE

One step means one name checked. Do it properly — do not estimate.

| FIND THIS NAME | LINEAR STEPS | BINARY STEPS | WHICH WON? |
|----------------|--------------|--------------|------------|
| ADAMS          |              |              |            |
| OWENS          |              |              |            |
| KHAN           |              |              |            |
| CHEN           |              |              |            |

Which name is the **only** one where linear search wins, and why? \_\_\_\_\_

## Task 7.2b • Show your halving

PROVE

Write out the binary search for **GRANT**. At each step, say which name you checked and which half you threw away.

| STEP | NAME CHECKED | HALF KEPT |
|------|--------------|-----------|
| 1    |              |           |
| 2    |              |           |
| 3    |              |           |
| 4    |              |           |

Roughly how many names were eliminated by the very first check? \_\_\_\_\_

# Why it matters at scale

16 MIN

## Task 7.3 · The doubling table

CALCULATE

Each time you double the list, binary search needs only **one** extra step. Fill in the gaps.

| LIST SIZE       | LINEAR · WORST CASE | BINARY · WORST CASE |
|-----------------|---------------------|---------------------|
| 15 names        | 15                  | 4                   |
| 30 names        |                     |                     |
| 60 names        |                     |                     |
| 1 000 names     |                     | 10                  |
| 1 000 000 names |                     | 20                  |

a. Going from 1 000 to 1 000 000 names multiplies the list by a thousand. How many extra steps does binary search need? \_\_\_\_\_

## Task 7.4 · The catch

THINK

a. Binary search is enormously faster. Why can you not always use it?

\_\_\_\_\_

b. Sorting a list takes real effort. Explain why it is still worth sorting a list you plan to search many times.

\_\_\_\_\_

c. You have a list of 1 000 names and you need to search it exactly **once**. Should you sort it first? Explain.

\_\_\_\_\_

\_\_\_\_\_

## Task 7.5 · The last word

THINK

In one sentence: why is bubble sort taught in schools even though no professional would ever use it?

\_\_\_\_\_

\_\_\_\_\_

## Before you hand this in

CHECKPOINT

Tick what you managed today

- ☐ I finished the main tasks
- ☐ I attempted the stretch task
- ☐ I checked my work with a partner

One thing I now understand that I did not this morning

\_\_\_\_\_

Teacher initials \_\_\_\_\_

# Answer key

Sorting and Searching by Hand

ANSWERS

## Task 7.1 • Bubble sort, every pass

Starting list: 8 3 7 1 5 2

| PASS | LIST AFTER THIS PASS | SWAPS    |
|------|----------------------|----------|
| 1    | 3 7 1 5 2 8          | 5        |
| 2    | 3 1 5 2 7 8          | 3        |
| 3    | 1 3 2 5 7 8          | 2        |
| 4    | 1 2 3 5 7 8          | 1        |
| 5    | 1 2 3 5 7 8          | 0 • stop |

**a.** Five passes — the fifth produces no swaps, which is how you know it is finished. **b.** Eleven swaps in total. A student who stops after pass 4 has the right list but has not proved it is sorted; give partial credit and explain the difference.

## Task 7.2 • Step counts

| NAME  | LINEAR | BINARY |
|-------|--------|--------|
| ADAMS | 1      | 4      |
| OWENS | 15     | 4      |
| KHAN  | 11     | 3      |
| CHEN  | 3      | 4      |

**ADAMS** is the only clear win for linear search, because it is the very first name. **CHEN** is also close. Accept small variations in the binary counts — the halving point can be taken slightly differently — as long as the working is shown.

## Task 7.3 • The doubling table

| SIZE      | LINEAR    | BINARY |
|-----------|-----------|--------|
| 15        | 15        | 4      |
| 30        | 30        | 5      |
| 60        | 60        | 6      |
| 1 000     | 1 000     | 10     |
| 1 000 000 | 1 000 000 | 20     |

**a.** Ten extra steps. The list grew a thousandfold; the work grew by ten.

## Task 7.2b • Binary search for GRANT, step by step

1. Check 8, HUANG. GRANT comes earlier → keep 1–7.
2. Check 4, DIAZ. GRANT comes later → keep 5–7.
3. Check 6, FOX. GRANT comes later → keep 7 only.
4. Check 7, GRANT. Found.

The first check eliminated **eight** names — over half the list — in a single step. That is the whole idea. Accept slightly different halving points provided the working is shown.

## Task 7.4 • Model answers

- The list has to be sorted first. On an unsorted list, throwing away half is meaningless — the name could be anywhere.
- Sorting is a one-off cost. Every search afterwards is cheap, so the effort is repaid many times over.
- No. Sorting 1 000 names costs far more than one linear search through them. Sort only when you will search repeatedly — the best question on the page, because it weighs cost against benefit.

## Task 7.5 • the closing question

# Bug Hunt

2 MINUTES

Students learn the three kinds of program bug, classify six real error messages, then find and fix six broken programs. The last one runs perfectly and still gives the wrong answer, which is the most important bug of all.

## You do not need to know this subject

Every bug is a single line, and the answer key names the line, the bug type and the fix for each one. You are checking answers against a key, not diagnosing code.

## Photocopy these pages only

## Pages 42–44

One per student. **Do not photocopy this page or the answer key.**

## What students need

A pen. Pairs are fine and productive for this lesson.

## WHAT TO SAY TO START

*“Every program you have ever used had bugs in it. Professional programmers spend more time fixing than writing. Today you get six broken programs and you have to work out what is wrong with each one.”*

## HOW THE LESSON RUNS

| TIME  | WHAT HAPPENS                           | WHAT YOU DO                                  |
|-------|----------------------------------------|----------------------------------------------|
| 0–5   | Hand out sheets, settle                | Use the door script on page 5                |
| 5–14  | Read the three bug types together      | The distinction matters more than the code   |
| 14–24 | Task 8.1 — classify the error messages | Six answers, all in the key                  |
| 24–42 | Task 8.2 — the six broken programs     | One bug each. Pairs work well here           |
| 42–52 | Task 8.3 — the program that lies       | Hardest and most important task on the sheet |
| 52–55 | Collect sheets                         | Discussion prompt below                      |

## Questions students will ask

### “The program looks fine to me.”

For Task 8.3 that is the correct reaction — it does look fine. Tell them to work out the answer by hand first, then compare.

### “Do I need to know Python?”

No. Every bug is visible from the plain-English comment beside the code. Say this before they start or half the class will not begin.

### “Is a spelling mistake a bug?”

In code, yes — a misspelt variable name is one of the commonest bugs there is. In a message shown to the user, it is a bug too, just a cosmetic one.

## If they finish early

Task 8.4 asks students to write a test plan — what values would you feed a program to prove it works. Strong students should be pushed towards boundary values: zero, negative numbers, and the exact number where behaviour changes.

## If the computers are working

Students can type each broken program in and read the real error message, which is far more convincing than the printed one. Ask them to predict the message before they run it.

## Curriculum focus · for the regular teacher's records

Testing and debugging; syntax, runtime and logic errors; developing test data. Discussion prompt: “A program that crashes tells you something is wrong. A program that quietly gives the wrong answer does not. Which would you rather have?”

# Three kinds of broken

14 MIN

NAME \_\_\_\_\_

CLASS \_\_\_\_\_

DATE \_\_\_\_\_

## 1 • Syntax error

The code breaks the rules of the language. It will not run at all.

```
print("hello"
```

Missing bracket. Nothing happens.

## 2 • Runtime error

The code is legal, but something impossible happens while it runs.

```
print(10 / 0)
```

Starts, then crashes.

## 3 • Logic error

It runs perfectly and gives the wrong answer. The dangerous one.

```
area = w + h
```

No error. Just wrong.

### Worked example • the same program, three ways to break it

```
# syntax
if mark ≥ 50
    print("PASS")
```

Missing colon. Will not run at all.

```
# runtime
mark = int("banana")
if mark ≥ 50:
```

Starts, then crashes on line 2.

```
# logic
if mark ≥ 5:
    print("PASS")
```

Runs perfectly. Passes everyone with 5 or more.

### Task 8.1 • Name that bug

CLASSIFY

Write **S** for syntax, **R** for runtime or **L** for logic beside each one.

| # | WHAT HAPPENED                                                      | S / R / L |
|---|--------------------------------------------------------------------|-----------|
| a | The program will not start. The editor highlights a missing colon. |           |
| b | It calculates the average of 5 numbers by dividing by 4.           |           |
| c | It crashes when the user types a word instead of a number.         |           |
| d | A variable name is spelt totl in one place and total in another.   |           |
| e | It prints every student twice.                                     |           |
| f | It stops when it tries to open a file that has been deleted.       |           |

# Six broken programs

18 MIN

Each program has **exactly one** bug. Find it, name the type, and write the fix.

bug 1

PYTHON

```
# Print a greeting five times
for i in range(5)
    print("Hello")
```

Type: \_\_\_\_\_

The fix:

---

---

bug 2

PYTHON

```
# Add up three marks
m1 = 10
m2 = 20
m3 = 30
total = m1 + m2 + m4
print(total)
```

Type: \_\_\_\_\_

The fix:

---

bug 3

PYTHON

```
# Ask for a number and double it
n = input("Number: ")
print(n * 2)

# User types 5, program prints 55
```

Type: \_\_\_\_\_

The fix:

---

bug 4

PYTHON

```
# Count down from 5 to 1
count = 5
while count > 0:
    print(count)
# runs forever
```

Type: \_\_\_\_\_

The fix:

---

bug 5

PYTHON

```
# Pass mark is 50 or above
if mark > 50:
    print("PASS")
else:
    print("FAIL")
```

Type: \_\_\_\_\_

The fix:

---

---

bug 6

PYTHON

```
# Print the numbers 1 to 10
for i in range(1, 10):
    print(i)

# only prints up to 9
```

Type: \_\_\_\_\_

The fix:

---

---

# The program that lies

16 MIN

## Task 8.3 · It runs. It is wrong.

HARDEST

This program calculates a class average. It never crashes. It always produces a number. That number is wrong.

average.py

PYTHON

```
marks = [60, 70, 80, 90]
total = 0

for m in marks:
    total = total + m

average = total / 3
print("Average:", average)
```

a. Work out the correct average by hand: \_\_\_\_\_

b. What does the program print instead? \_\_\_\_\_

c. Which line is wrong? \_\_\_\_\_

d. Write the corrected line:

\_\_\_\_\_

e. Why is this kind of bug more dangerous than a crash?

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

## Task 8.4 · Write a test plan

EXTENSION

A program takes a student's mark and prints PASS if it is 50 or more. List five values you would test it with, and what each one proves.

TEST VALUE

WHAT IT PROVES

|  |  |
|--|--|
|  |  |
|  |  |
|  |  |
|  |  |
|  |  |
|  |  |

## Before you hand this in

CHECKPOINT

Tick what you managed today

- ☐ I finished the main tasks
- ☐ I attempted the stretch task
- ☐ I checked my work with a partner

One thing I now understand that I did not this morning

\_\_\_\_\_

Teacher initials \_\_\_\_\_

# Answer key

## Bug Hunt

ANSWERS

### Task 8.1 • Classification

- a. **S** — missing colon, will not run
- b. **L** — runs fine, answer wrong
- c. **R** — legal code, crashes on bad input
- d. **S** or **R** — accept either, with reasoning
- e. **L** — no error, wrong output
- f. **R** — crashes while running

### About question d

A misspelt variable is often reported as a runtime error rather than a syntax one, because the program starts before it discovers the name does not exist. Both answers are defensible; the reasoning is what you are marking.

### Task 8.2 • The six bugs

| # | TYPE   | THE BUG                                     | THE FIX                                        |
|---|--------|---------------------------------------------|------------------------------------------------|
| 1 | Syntax | No colon after the for line                 | <code>for i in range(5):</code>                |
| 2 | Syntax | m4 does not exist; should be m3             | <code>total = m1 + m2 + m3</code>              |
| 3 | Logic  | input gives text, so "5" doubled is "55"    | <code>n = int(input("Number: "))</code>        |
| 4 | Logic  | count never changes, so the loop never ends | <code>count = count - 1</code> inside the loop |
| 5 | Logic  | A mark of exactly 50 fails                  | <code>if mark ≥ 50:</code>                     |
| 6 | Logic  | range stops before 10                       | <code>for i in range(1, 11):</code>            |

Bugs 5 and 6 are both **off-by-one** errors, the single most common logic bug in programming. Worth naming out loud — students remember the phrase.

### Task 8.3 • The program that lies

- a.  $(60+70+80+90) \div 4 = 300 \div 4 = 75$
- b.  $300 \div 3 = 100$
- c. The line average = `total / 3`
- d. average = `total / len(marks)` — accept `total / 4`, but point out that it breaks the moment a fifth mark is added. That distinction is worth a mark on its own.
- e. A crash tells you immediately that something is wrong. This program reports a confident, plausible number — 100 is a believable average — so nobody investigates. Wrong answers that look right can go unnoticed for years.

### Task 8.4 • A good test plan

| VALUE                 | WHAT IT PROVES                                                  |
|-----------------------|-----------------------------------------------------------------|
| 49                    | Just below the boundary — should FAIL                           |
| 50                    | Exactly on the boundary — should PASS                           |
| 51                    | Just above the boundary — should PASS                           |
| 0                     | The lowest sensible value                                       |
| 100 or -5 or "banana" | The top of the range, or invalid input the program must survive |

# What AI Can and Cannot Do

2 MINUTES

Students find out what an AI chatbot is actually doing when it answers, sort tasks into safe and risky uses, examine a confident paragraph that cannot be verified, and write their own rules for using AI in schoolwork.

## You do not need to know this subject

This is a discussion and judgement lesson. There is no technology involved and no correct opinion — the answer key gives you the reasoning to look for rather than a single right answer.

## Photocopy these pages only

### Pages 47–49

One per student. **Do not photocopy this page or the answer key.**

## What students need

A pen. No devices needed, even if the lesson is about them.

## WHAT TO SAY TO START

*"An AI chatbot does not know anything. It predicts what words usually come next. That sounds like a small difference. Today you will find out why it changes everything about how you should use it."*

## HOW THE LESSON RUNS

| TIME  | WHAT HAPPENS                              | WHAT YOU DO                                        |
|-------|-------------------------------------------|----------------------------------------------------|
| 0–5   | Hand out sheets, settle                   | Use the door script on page 5                      |
| 5–14  | Read 'What it is actually doing' together | The autocomplete comparison does the heavy lifting |
| 14–26 | Task 9.1 — sorting tasks by risk          | Expect disagreement. Ask for reasons, not votes    |
| 26–40 | Task 9.2 — the confident paragraph        | The point is that none of it can be checked        |
| 40–50 | Task 9.3 — is it cheating?                | The most valuable discussion in the pack           |
| 50–60 | Task 9.4, collect sheets                  | Discussion prompt below                            |

## Questions students will ask

### "Is using AI cheating?"

The honest answer is that it depends entirely on what was asked of you. Task 9.3 is designed to make students argue that out rather than be told.

### "Does AI know it is wrong?"

No. It has no way to tell the difference between a true sentence and a plausible one. That is the single most important idea in the lesson.

### "Do teachers use AI?"

Many do, for planning and drafting. Being straightforward about that earns far more credibility than pretending otherwise.

## If they finish early

Task 9.4 asks students to write five rules for AI use at school. Strong students should then be asked which of their own rules would be impossible to enforce — that is a harder and more honest question than writing the rules.

## If the computers are working

If AI tools are permitted at your school, students can ask a chatbot the same factual question three times and compare the answers. Inconsistency between the three is the clearest possible demonstration of the point.

## Curriculum focus • for the regular teacher's records

Impacts of digital systems on society; evaluating information sources; ethical use of technology; data and bias. Discussion prompt: *"If an AI is confident and wrong, and you cannot tell the difference, whose fault is the mistake?"*

# It is predicting, not knowing

16 MIN

NAME \_\_\_\_\_

CLASS \_\_\_\_\_

DATE \_\_\_\_\_

## The most useful thing to understand about AI chatbots

When your phone suggests the next word in a message, it is not reading your mind. It has seen millions of messages and it knows which word usually follows.

An AI chatbot is doing the same thing, on an enormous scale. It produces the words that most plausibly come next. It is extraordinarily good at this – but **plausible is not the same as true**, and the system has no way of telling the two apart.

## Task 9.1 · Sort these by risk

JUDGE

Tick one column for each task, then write your reason.

| TASK                                                 | SAFE | RISKY | WHY |
|------------------------------------------------------|------|-------|-----|
| Suggesting ideas for a story                         |      |       |     |
| Telling you the date of a historical event           |      |       |     |
| Rewriting your paragraph more clearly                |      |       |     |
| Giving medical advice about a symptom                |      |       |     |
| Explaining a concept you did not understand in class |      |       |     |
| Writing your assignment for you                      |      |       |     |
| Summarising a page you have already read             |      |       |     |
| Naming the sources for a research task               |      |       |     |

Look at everything you marked **safe**. What do those tasks have in common? \_\_\_\_\_

# Confident and unverifiable

18 MIN

## An AI was asked to write about an Australian inventor

Read this carefully. It is well written, specific and completely confident.

*Margaret Ellwood Harrow was born in Ballarat in 1897 and is widely credited with inventing the rotary seed drill. After studying at the Ballarat School of Mines, she filed her first patent in 1923 and by 1931 her design was in use on more than 4 000 farms across Victoria and South Australia. The Harrow Drill Company employed 212 people at its peak. She was awarded the Kernot Medal in 1948, and a street in Ballarat East still bears her name.*

Every sentence sounds like a fact. Every sentence has a specific detail attached. That is exactly what makes it convincing.

## Task 9.2 · What would you have to check?

INVESTIGATE

You are not being asked whether this is true. You are being asked what you would have to **check**, and how.

| # | A CLAIM THAT NEEDS CHECKING | HOW WOULD YOU CHECK IT? |
|---|-----------------------------|-------------------------|
| 1 |                             |                         |
| 2 |                             |                         |
| 3 |                             |                         |
| 4 |                             |                         |
| 5 |                             |                         |

Which single detail in the paragraph would be **fastest** to check, and why? \_\_\_\_\_

### The point of this task

A wrong answer that sounds uncertain is easy to catch. A wrong answer delivered with names, dates and numbers is not. Confidence is not evidence.

# Whose work is it?

18 MIN

## Task 9.3 · Cheating, or not?

DECIDE

Mark each one **OK**, **NOT OK** or **DEPENDS**, then give your reason. There is genuine disagreement about some of these.

| # | WHAT THE STUDENT DID                                         | VERDICT | REASON |
|---|--------------------------------------------------------------|---------|--------|
| a | Asked AI to explain a topic, then wrote the essay themselves |         |        |
| b | Asked AI to write the essay, then changed a few words        |         |        |
| c | Used AI to check their spelling and grammar                  |         |        |
| d | Asked AI for five ideas, picked one, researched it properly  |         |        |
| e | Used AI to summarise a book they were meant to read          |         |        |

## Task 9.4 · Write the rules

EXTENSION

Write three rules for using AI on schoolwork that you think are **fair** — rules you would accept even when they worked against you.

1. \_\_\_\_\_
2. \_\_\_\_\_
3. \_\_\_\_\_

Now the hard part: which of your three rules would be **impossible for a teacher to enforce**, and does that make it a bad rule?

---



---



---

## Before you hand this in

CHECKPOINT

### Tick what you managed today

- ☐ I finished the main tasks
- ☐ I attempted the stretch task
- ☐ I checked my work with a partner

### One thing I now understand that I did not this morning

\_\_\_\_\_

Teacher initials \_\_\_\_\_

# Answer key

What AI Can and Cannot Do

ANSWERS

## Task 9.1 • What to look for

There is no fixed answer key here — mark the reasoning. The pattern you are steering towards:

**Generally safe** — tasks where **you** already know the material and can spot a mistake: generating ideas, rewording your own writing, summarising something you have read, explaining a concept you can then check against your notes.

**Generally risky** — tasks where you **cannot tell** whether the answer is right: historical dates, medical advice, naming sources, and anything you are supposed to have produced yourself.

**The common thread:** AI is safest where you are able to check it. That is the sentence to draw out of the class.

## Task 9.2 • Important

**Margaret Ellwood Harrow does not exist.** The paragraph is entirely invented — the person, the company, the patent, the medal and the street. Tell the class this at the end, not the beginning. The uncomfortable moment when they realise how convincing it was is the lesson.

## Task 9.2 • Marking the checks

Any five specific claims: the birth year and place • the invention of the rotary seed drill • the 1923 patent • the figure of 4 000 farms • the 212 employees • the Kernot Medal in 1948 • the street name.

Good checking methods: search a patent register • check the official list of medal recipients • look at a map or street directory • search a library or newspaper archive • look for any **independent** source that is not simply repeating the same paragraph.

**Fastest to check:** the street name — an online map answers it in seconds. Award full marks for any answer that picks the claim with a single, public, searchable record behind it.

## Task 9.3 • Where classes usually land

- a. **OK** — the thinking and the writing are the student's. This is using AI as a tutor.
- b. **NOT OK** — near-universal agreement. The work is not theirs.
- c. **OK** — no different from a spellchecker, which nobody objects to.
- d. **OK** — ideas are not the assessed part; the research is.
- e. **DEPENDS**, and this is the best argument on the page. If the task was to *read* the book, a summary defeats it entirely. If the book was background for something else, it may be perfectly reasonable. Push the class to name **what was actually being assessed** — that is the principle underneath every one of these.

## Task 9.4 • The enforcement question

Most student rules are unenforceable, and that is fine. An unenforceable rule can still be a good one if people agree with it — most school rules work that way. Accept any answer that separates **whether a rule is right** from **whether it can be policed**.

# Your Digital Footprint

2 MINUTES

Students examine a fictional student's public posts and work out exactly how much a stranger could learn from them. It is genuinely unsettling in a productive way, and it needs no technology at all.

## You do not need to know this subject

Everything here is reading and judgement. The fictional profile is on the student page and the answer key lists every piece of information that can be pieced together from it.

## Photocopy these pages only

### Pages 52–54

One per student. **Do not photocopy this page or the answer key.**

## What students need

A pen. Do not ask students to bring up their own real accounts in class.

## WHAT TO SAY TO START

*"Nobody posts their address online. But six ordinary posts, none of them careless on its own, can add up to your address, your school, and the exact times your house is empty. Today you are going to prove that."*

## HOW THE LESSON RUNS

| TIME  | WHAT HAPPENS                           | WHAT YOU DO                                            |
|-------|----------------------------------------|--------------------------------------------------------|
| 0–5   | Hand out sheets, settle                | Use the door script on page 5                          |
| 5–12  | Read the three-part panel together     | The third part surprises them most                     |
| 12–28 | Task 10.1 — the profile audit          | The key lists everything findable. Let them hunt first |
| 28–40 | Task 10.2 — rating information by risk | Expect argument about phone numbers and school names   |
| 40–50 | Task 10.3 — the future test            | Answer key has guidance                                |
| 50–55 | Task 10.4, collect sheets              | Discussion prompt below                                |

## Questions students will ask

### "None of that is private though."

Correct — that is precisely the point. No single post is careless. The risk comes from combining them, which is what Task 10.1 makes them do.

### "Should I delete my accounts?"

No, and avoid saying so. The lesson is about what you post and who can see it, not about withdrawing from the internet.

### "Can you really find someone from a photo?"

Often, yes — uniforms, street signs, bus numbers and shopfronts are all in the background of ordinary photos. Keep this general rather than demonstrating it.

## If they finish early

Task 10.4 asks students to write advice for a Year 7 student. Strong students can then argue the opposite case: what are the genuine costs of having no online presence at all when you apply for a job or a course?

## If the computers are working

Students can search their own name in a browser they are not signed in to, and see what comes back. Set clear boundaries first: their own name only, and no searching for classmates.

## Curriculum focus · for the regular teacher's records

Digital footprints and personal data; privacy and security of personal information; responsible use of digital systems. Discussion prompt: "You can delete a post. Can you delete a screenshot somebody else took of it?"

# Six harmless posts

12 MIN

NAME

CLASS

DATE

## What you post

Photos, comments, videos. The part you control — and the part people think is the whole picture.

## What others post

Tagged photos, group chats, a friend naming your school. You did not choose any of it.

## What is simply recorded

Locations, times, which device, what you searched. Nobody posted it. It exists anyway.

## Public posts from one account · all visible to anyone

**MON 7:52am**

Bus is late AGAIN. Third time this week  
photo shows a bus stop and route 512 sign

**MON 4:10pm**

Training til 6 every Monday and Wednesday. Dead.  
photo shows a sports oval and a school jumper with a visible crest

**WED 8:30pm**

Happy birthday to my absolute legend of a mum!! 47 today  
photo taken inside a kitchen

**FRI 3:40pm**

Anyone else at Northgate Plaza after school? Im at the food court

**SAT 11:15am**

Whole family away next week, cant wait  
photo shows four packed suitcases in a hallway

**SUN 9:00pm**

New phone!! Finally  
photo shows the box, and a school diary on the desk behind it with a name label

# What can a stranger work out?

18 MIN

## Task 10.1 · The audit

INVESTIGATE

Using only the six posts on the previous page, work out what a stranger could learn. Write the post that gives it away.

### WHAT A STRANGER COULD LEARN

### WHICH POST GIVES IT AWAY?

Which school they attend

Roughly which suburb they live in

Their full name

Two afternoons they are definitely not home

A whole week the house will be empty

Their mother's year of birth

Something valuable that is in the house

Where to find them on a Friday afternoon

Which single post is the most dangerous, and why? \_\_\_\_\_

## Task 10.2 · Rate the risk

JUDGE

Rate each one from 1 (harmless) to 5 (never post this).

### INFORMATION

1-5

Your first name

Your school

Your suburb

Your date of birth

### INFORMATION

1-5

Your weekly routine

When your house is empty

A photo in school uniform

Your phone number

Pick any two you rated low on their own. Explain how they become dangerous **together**.

# The long memory

16 MIN

## Task 10.3 · Would this matter in eight years?

DECIDE

Imagine each of these is found by an employer, a university, or a sports club selector when you are 23. Mark it **fine**, **awkward** or **damaging**, and say why.

| # | THE POST                                               | VERDICT | WHY |
|---|--------------------------------------------------------|---------|-----|
| a | A photo of you winning a school competition            |         |     |
| b | An argument in the comments where you insulted someone |         |     |
| c | A joke about a teacher, using their real name          |         |     |
| d | A strong political opinion you no longer hold          |         |     |
| e | A photo of you at a party, tagged by someone else      |         |     |

## Task 10.4 · Advice for a Year 7

EXTENSION

Write three pieces of advice for a Year 7 student getting their first phone. Make them realistic — “never post anything” is advice nobody follows.

1. \_\_\_\_\_
2. \_\_\_\_\_
3. \_\_\_\_\_

## Task 10.5 · The last word

THINK

You can delete a post. Explain in one sentence why that does not necessarily mean it is gone.

---



---

## Before you hand this in

CHECKPOINT

### Tick what you managed today

- ☐ I finished the main tasks
- ☐ I attempted the stretch task
- ☐ I checked my work with a partner

### One thing I now understand that I did not this morning

---

Teacher initials \_\_\_\_\_

# Answer key

Your Digital Footprint

ANSWERS

## Task 10.1 • Everything findable in six posts

| WHAT A STRANGER LEARNS    | FROM WHICH POST        | HOW                                                             |
|---------------------------|------------------------|-----------------------------------------------------------------|
| The school                | Monday 4:10pm          | School crest visible on the jumper                              |
| The suburb                | Monday 7:52am + Friday | Bus route 512 and Northgate Plaza together narrow it right down |
| Their full name           | Sunday 9:00pm          | Name label on the school diary in the background                |
| Two afternoons away       | Monday 4:10pm          | Training every Monday and Wednesday until 6                     |
| A week the house is empty | Saturday 11:15am       | "Whole family away next week" plus packed suitcases             |
| Mother's birth year       | Wednesday 8:30pm       | Age 47 and the date of the post                                 |
| Something valuable inside | Sunday 9:00pm          | A brand new phone, shown in its box                             |
| Friday whereabouts        | Friday 3:40pm          | Named the shopping centre and the food court                    |

**The most dangerous post is Saturday.** It publicly announces an empty house for a known week, to an audience that already knows the suburb from earlier posts. On its own it is harmless. Combined with the other five it is an invitation. That combination is the entire lesson.

## Task 10.2 • Guidance

Ratings are a judgement, not a key. Look for reasoning. The usual pattern:

Low on their own: first name, suburb, a school photo.

High on their own: phone number, when the house is empty, full date of birth.

**The combination question is the one that matters.** School + routine gives someone a time and a place to find a specific child. Neither piece is alarming alone.

## Task 10.3 • What to draw out

**a** is fine and worth pointing out — a footprint can help you. **b** and **c** are damaging because they show how someone treats other people. **d** is the interesting one: people are allowed to change their minds, but the internet does not record that they did. **e** raises the real problem — **you did not post it and you cannot delete it.**

## Task 10.4 and the closing question

**Good advice looks like:** check what a post shows in the background, not just the subject • wait ten minutes before posting when angry • never announce that you are away until you are home • assume a private post can be screenshotted • review who can see your account once a term.

**The closing question:** deleting removes your copy, not everyone else's. Screenshots, archived pages, and other people's reposts all survive. Accept any answer reaching "someone else may already have a copy".

# Handover note

90 SECONDS

Photocopy this page or fill it in directly. Ninety seconds of your time saves the regular teacher a great deal of guesswork.

SUBSTITUTE TEACHER  
\_\_\_\_\_CLASS  
\_\_\_\_\_DATE  
\_\_\_\_\_PERIOD  
\_\_\_\_\_**Lesson used**

TICK ONE

- |                                                              |                                                                  |
|--------------------------------------------------------------|------------------------------------------------------------------|
| <input type="checkbox"/> 01 • Follow My Instructions Exactly | <input type="checkbox"/> 06 • Sending a Message Across the World |
| <input type="checkbox"/> 02 • Counting Like a Computer       | <input type="checkbox"/> 07 • Sorting and Searching by Hand      |
| <input type="checkbox"/> 03 • Trace the Code                 | <input type="checkbox"/> 08 • Bug Hunt                           |
| <input type="checkbox"/> 04 • Flowchart Detective            | <input type="checkbox"/> 09 • What AI Can and Cannot Do          |
| <input type="checkbox"/> 05 • How Attacks Actually Work      | <input type="checkbox"/> 10 • Your Digital Footprint             |

**How far did the class get?**

- ☐ Everyone finished the main tasks
- ☐ Most finished; some are partway
- ☐ We got through about half
- ☐ We did not get far — see notes

**Sheets**

- ☐ Collected and left on the desk
- ☐ Students kept them
- ☐ Marked against the answer key

**Anything the regular teacher should know**

NOTES

---

---

---

---

---

**Students to mention • for any reason, good or otherwise**

NAMES

---

---

---

---

**Thank you**

Walking into an unfamiliar subject is not easy. Leaving a clear note behind is the part that makes the next teacher's day work, and it is genuinely appreciated.

# Licence & what else is available

## Terms of use

Licensed for **one teacher in one classroom**. You may print and photocopy these pages for the students you personally teach, and you may pass the pack to a substitute teacher covering your own classes.

You may not redistribute it to colleagues, upload it to a school intranet or shared drive, resell it, or post it publicly. Additional licences for a faculty or whole school are available at a discount.

© Digital Vector · digitalvector.com.au

## Curriculum alignment

Australian Curriculum and Victorian Curriculum, Digital Technologies Years 7–10. Across the ten lessons the pack touches:

- Representing data in binary and other forms
- Designing and tracing algorithms
- Reading and debugging programs
- How networks transmit and route data
- Security, privacy and personal data
- Impacts of digital systems, including artificial intelligence

Individual lesson teacher pages name the specific focus for that lesson.

## Also from Digital Vector

### Physical Computing with the Raspberry Pi 5

Labs 1–3: build and code a circuit, a traffic light and a pedestrian crossing.

Print-ready, full teacher answers.

### AI Literacy for Secondary Students

Classroom-ready material on how AI works, where it fails and how to use it honestly in schoolwork.

**digitalvector.com.au**

## Found an error, or want a lesson on something else?

Feedback is genuinely read. If a lesson did not land with your class, or you need a sub plan on a topic that is not here, get in touch through [digitalvector.com.au](https://digitalvector.com.au) and it will be considered for the next update.

EMERGENCY SUB PLANS PACK

## Digital Technologies • Years 7–10

Ten lessons • No preparation • No computers required